SITAA Contactless Fingerprint SDK

MinutiaeNet Fine-Tuning & Data Collection Plan

Path to >85% Matching Accuracy — Extraction-Level Root Cause & Staged Data Strategy

Project SU-C1-2025-02 · YellowSense Technologies
UIDAI SITAA Challenge
Detailed execution plan for the extraction / fine-tuning lever of the accuracy-improvement effort
2026-08-25

This document lays out the full plan to close the gap from the current measured 52.4% matching accuracy to the mandatory >85% acceptance-gate target: why MinutiaeNet is the right model to fine-tune rather than replace, the four execution stages (ground-truth labelling, dataset preparation, training and testing), a staged data-collection strategy scaling to 1000+ images across 150–300 people, and the final testing phase against the acceptance gate.

Context & Current State

Metric	Value
Matching accuracy (TAR)	52.4%
FRR	47.6%
FAR	1.7%
Algorithm / threshold	legacy / 0.23 — already bake-off'd across 10 algorithms, 3 real-data rounds
Target (mandatory acceptance gate)	>85%, ≥20 users, contactless-to-contactless

The matching algorithm and threshold are close to maxed out: **legacy** won a 10-algorithm bake-off across three independent real-data rounds, and the threshold independently re-derives to ~0.23 every round. A real experiment this cycle found MinutiaeNet is tightly coupled to the current preprocessing's specific output — swapping the enhancement stage alone made self-consistency *worse*, not better. That rules out isolated pipeline-stage swaps and points at the extractor itself: MinutiaeNet's own reported score is F1=0.710 — roughly 30% of minutiae are noise — and everything downstream (matcher, threshold, algorithm choice) inherits that noise. **Confirmed, not just suspected:** IIT Bombay has since confirmed directly that MinutiaeNet was trained on **PolyU**, a contact-scanner fingerprint database — not contactless. The domain-shift diagnosis this whole plan rests on is now a fact, not an inference.

Decision: Fine-Tune, Not Train From Scratch

Fine-tune the deployed model, warm-started from `best_f1.pth`. Note on architecture, corrected from an earlier assumption: this is *not* a direct port of the public CoarseNet+FineNet MinutiaeNet architecture. Cloning the original reference repository and comparing it against this project's own `detect_minutiae()` shows the real CoarseNet takes a 400×400 input and outputs up to 10 maps (its own segmentation, dual orientation fields, Gaussian-encoded position offsets); the deployed model takes a 256×256 input and returns exactly 4 tensors (location, cos, sin, type). `best_f1.pth` is a simplified, custom single-network reimplementation — “in the style of” MinutiaeNet, not a weights-compatible port. The fine-tuning logic still holds: it's a CNN that already encodes minutiae domain knowledge (ridge/bifurcation geometry), now confirmed trained on contact-scanner (PolyU) images — a domain-shift problem, which fine-tuning is the standard fix for. What changes is that its original training code cannot be reused as-is (see Stage B).

Training a new architecture from scratch was considered and deliberately deferred: no training infrastructure exists yet for any model in this repository, and it needs an order of magnitude more labeled data to converge — against a short PoC deadline. Revisit only if this plan's fine-tuning results measurably plateau below what the gate needs; the labeled dataset built here is directly reusable if that happens (see Contingency, below).

Target

1000+ real images across 150–300 distinct people, collected in stages, with a fine-tune-and-measure cycle run at each stage — not one upfront collection push followed by a single training run. At ~5 images per person (1 enrollment + 4 independent test captures across ≥2 separate sessions, varying lighting/angle/distance), 200 people ≈ 1,000 images, 300 people ≈ 1,500 images.

Staging is the point: it turns “did more data help” into a measured curve instead of a guess, and it means there is an improving, testable model at every checkpoint, not just at the end.

Data Collection Protocol

- **Per person:** 1 enrollment capture + 4 test captures minimum, across ≥2 separate sessions (not all in one sitting).
- **Vary deliberately:** lighting, distance, angle, day — this is exactly the robustness the accuracy gate needs.
- **Vary skin tone deliberately too.** Now that PolyU (contact-scanner) is confirmed as MinutiaeNet's training domain, this matters specifically: a scanner reads ridges via pressure, not light — skin tone is irrelevant to it. A phone camera reads ridges via reflected light, where skin tone and reflectance matter directly. This is a contactless-only failure mode the model has zero training exposure to; don't collect a demographically narrow sample by accident.
- **Consent required; no real Aadhaar numbers.**
- **Yield risk:** the last real testing round attempted 30 people and only 12 were usable after quality gating (40% yield). At that rate, hitting 150–300 *usable* people could mean attempting 375–750. Mitigate by (1) fixing real-time capture guidance at collection time (enforce an actual distance range, consistent enroll/test framing) and (2) planning collection targets with a buffer above the usable target rather than assuming 100% yield.

STAGE A — Ground Truth Labelling

Goal: get a second, independent opinion on every image so the fine-tune trains on trustworthy points, not MinutiaeNet's own uncorrected noise.

- **Set up MINDTCT** (NIST NBIS's open-source minutiae extractor) under WSL or Docker — the Windows-native build is the historically painful route, avoid it. This is scriptable/batchable, unlike NIST's FpMV viewer (GUI-only, no batch mode — useful only for spot-checking individual confusing images by eye).
- **Common coordinate space:** run every image through `fir.py`'s existing 500 DPI ridge-frequency normalization (already built for the FIR export) before feeding it to MINDTCT. Scale MinutiaeNet's output points into that same coordinate space (the pipeline already scales its points back to original image size — one more scale step, not new machinery).
- **Feed the two extractors different preprocessing depths, deliberately.** MinutiaeNet gets the *full* pipeline output, including the adaptive-threshold binarization step — that step is very likely the hand-engineered bridge that makes contactless photos resemble what MinutiaeNet's PolyU-trained weights expect, so don't remove it. MINDTCT should instead get the segmented+enhanced grayscale image *before* that binarization step — MINDTCT has its own internal binarization designed for raw-ish scanner-style input, and double-binarizing would distort its output in ways that look like disagreement but are really just a format mismatch. Keep both in the same crop/coordinate frame so the consensus merge below still lines up.
- **Angle convention:** convert MINDTCT's direction output to the project's `atan2(sin, cos)` radians convention before comparing.
- **Consensus merge:** for each MinutiaeNet point, look for a MINDTCT point within a position tolerance (start ~10–15px, tune by eye) and direction tolerance (~±20–30°). Both agree → high-confidence label. Only one side has it → mark “disputed,” don't guess.
- **This consensus is weaker than it first looks — plan around that.** MINDTCT was also built and tuned for contact-based scans, not phone photos. Two extractors agreeing on a contactless image isn't independent confirmation — they can share the same blind spot (e.g. both missing a point obscured by a lighting shadow neither was trained to expect). This makes the hand-corrected gold set the only genuinely independent signal in the pipeline, not just a spot-check.
- **Hand-correct only the held-out gold/validation slice** (see Stage B) — add missed points, delete spurious ones, fix directions. Brief whoever does this to distrust “textbook contact-print” instincts — contactless artifacts (reflectance shadows, uneven lighting) look different. The one genuinely manual step in the pipeline.

STAGE B — Dataset Preparation

- **Build the training-target generator** (does not exist in this repository yet): given an image + its consensus-labelled minutiae, produce the four maps the deployed model's loss trains against (location heatmap, cos map, sin map, type map) — the inverse of what `detect_minutiae` currently reads out. Checked directly: the original MinutiaeNet authors did publish their training code (MIT licensed), but it doesn't transfer — it's built for the different 10-output CoarseNet+FineNet architecture, on a Python 2.7 / TensorFlow 1.7 / Keras 2.1.6 stack. What *is* reusable: their Gaussian-smoothed heatmap encoding for minutia location (soft target, not a hard binary blob), their class-imbalance loss weighting for the location head (lamb_pos=10 : lamb_neg=0.5 — a real starting point instead of guessing), and an orientation-coherence regularizer (nearby predicted directions pulled toward agreement) — all worth adapting to the deployed model's actual 4-head format.
- **Train on images that passed through the exact same on-device preprocessing pipeline used at inference** (crop → segment → enhance → adaptive threshold) — not raw photos, and not a cleaner alternative preprocessing. That pipeline is very likely functioning today as a hand-engineered bridge between contactless captures and what the PolyU-trained weights expect; this is also why the Tier-0 classical-enhancement experiment made things worse, not better — it broke that bridge.
- **Disputed points get a "don't-care" loss mask**, not a confirmed-empty label — don't train the model to actively unlearn points one extractor merely didn't confirm.
- **Split by person, never by image** — a person's enrollment and test photos stay entirely on one side of the train/validation split, or validation numbers become meaningless.
- **Gold/validation set is capped at a fixed size** (~150–250 images, curated for diversity) even as the training pool keeps growing with each collection batch. Past a few hundred images, more validation data has diminishing return for *measuring* accuracy, while training data keeps benefiting from more — this also caps the manual-correction workload instead of letting it scale linearly forever.
- **Augment aggressively:** rotation, brightness/contrast jitter, slight scale. Even at the 1000+ image target this is a small CNN fine-tuning set; augmentation is not optional.

STAGE C Training (Fine-Tuning)

- Load `best_f1.pth` into the existing model class definitions the pipeline already uses for inference (the deployed 4-head architecture, not the original public CoarseNet+FineNet — see the Decision note above).
- **Freeze the backbone first as the opening experiment**, fine-tune only the output heads at a low learning rate (1e-5 to 1e-4) — cheap, fast signal. But budget for unfreezing deeper layers as the *likely* next step, not a rare fallback: PolyU vs. contactless is a full acquisition-modality gap (scanner pressure/controlled lighting vs. natural photo lighting on curved skin), which is exactly the kind of difference low-level convolutional layers encode, not just the output heads.
- Adapt the reusable pieces identified in Stage B — Gaussian heatmap encoding, the 10:0.5 imbalance weighting, an orientation-coherence term — into a multi-task loss for the deployed model's actual 4-head output, rather than assuming the original repo's loss functions apply directly.
- **K-fold across the training-pool people** at early milestones (small people counts make a single fixed split unreliable); a simple held-out split becomes reasonable once the pool is large enough (roughly by M2/M3).
- At later milestones, **warm-start each re-fine-tune from the previous milestone's best checkpoint**, not from `best_f1.pth` every time.

STAGE D Testing / Validation

Two tiers, in order:

- **Extraction-level:** precision/recall/F1 of each checkpoint against the gold set. Cheap, fast, catches obviously broken checkpoints before spending a real matching run on them.
- **System-level (the one that actually matters):** swap a promising checkpoint into the real pipeline, run `labtool/benchmark.py` against the current cumulative real dataset, get fresh TAR/FRR/FAR and genuine-vs-impostor score separation.
- **Re-calibrate matching algorithm + threshold from scratch** on the new extractor's output — legacy/0.23 were tuned to the old extractor's noise pattern; don't assume they still win.
- **Pick the deployed checkpoint by real matching accuracy**, not training loss or extraction F1 alone.
- Run `backend/tests/` (especially `test_separation_survives_partial_minutiae_overlap`) between iterations as a cheap regression check before spending a real-data round.
- Check impostor pairs specifically, not just genuine-pair accuracy, at every milestone.

Per-Batch Pipeline (Stages A→D Repeat at Every Milestone)

1. Run consensus labelling (Stage A) on the new batch; merge into the growing labelled pool.
2. Update the training pool with the new batch's people; keep the gold set fixed in size unless deliberately refreshing it for diversity.
3. Re-run fine-tuning (Stage C), warm-started from the previous milestone's checkpoint after M1.
4. Re-run testing (Stage D) against the current cumulative real dataset.
5. Record the point on the accuracy-vs-data-size curve: cumulative people, cumulative images, TAR/FRR/FAR.
6. Decide: continue to the next milestone, or stop early if the curve has visibly plateaued.

Milestones

Milestone	Cumulative people	Cumulative images	Purpose
M1	~100	~500	First trustworthy signal — is fine-tuning working at all
M2	~200	~1,000	Meaningful gate-progress zone — first checkpoint where an 85% claim becomes plausible
M3	~300	~1,500	Upper target — final checkpoint before the last testing phase, unless the curve plateaus earlier

Final Testing Phase

After M3, or wherever the curve plateaus, run one more full-rigor pass, matching the standard already set by the existing Internal Testing Annexure:

- Full cohort accounting — who was usable, who wasn't, and why; nothing silently dropped.
- TAR/FRR/FAR and genuine-vs-impostor score separation on the full cumulative dataset.
- Algorithm + threshold recalibrated from scratch against the final extractor.
- Explicit stated result against the ≥85%, ≥20-user, cL2cL gate — pass/fail stated plainly, not implied.

Success Criteria / Decision Points

- Define the go/no-go bar **before M1**: continue collecting only if each milestone shows a measurable TAR lift without FAR regressing past an agreed limit.
- If the curve plateaus below 85% before M3, that is the signal to reconsider the extractor architecture itself rather than continuing to collect data past the point it's helping — see Contingency, below.

Contingency: LEADER

If the fine-tuning pilot plateaus, the named fallback is LEADER (Cappelli & Ferrara, arXiv:2602.15493, Feb 2026) — a real, verified paper, not a rumor: same Cappelli/Ferrara pairing already correctly cited in this project's own `matching.py` for MCC. 0.9M parameters (vs. the deployed model's much larger footprint), 15ms GPU / 322ms CPU inference — small and fast enough to be a genuine on-device candidate, which the current extractor isn't.

Two caveats worth keeping in view, so this isn't mistaken for a shortcut: LEADER's own paper trains and evaluates it on plain and latent *contact* prints only — no contactless evaluation at all, meaning it has the same fundamental domain gap as the deployed model for this specific problem. And no public pretrained weights are confirmed available (checked directly against the paper and its referenced GitHub repo) — adopting it would mean training on the same labeled contactless dataset this plan already builds, not skipping that work.

Honest Caveats

No guarantee this reaches 85% — the curve may plateau earlier. Measuring at every milestone is what turns that risk into an early, cheap signal instead of a late, expensive surprise.

150–300 real, consenting people is a genuine logistical undertaking, not just an engineering task.

Full Ordered Execution Checklist

1. Set up mindtct (NBIS) under WSL/Docker; verify it runs on one sample image.
2. Fix real-time capture guidance (distance range, consistent enroll/test framing) to raise quality-gate yield before scaling up collection.
3. Begin data collection toward M1 (~100 people), following the per-person protocol.
4. Run all currently-available real images through fir.py's 500 DPI normalization.
5. Run MinutiaeNet on the full pipeline output (post-binarization) for each image; scale output points into that coordinate space.
6. Run mindtct on the pre-binarization (segmented+enhanced) version of each image; convert its angle convention.
7. Write and run the consensus-merge script; tag points agreed/disputed per image.
8. Select and hand-correct the fixed-size gold/validation set (~150–250 images, curated for diversity), split by person from the training pool.
9. Build the training-target generator (image + agreed minutiae → location/cos/sin/type maps), with a don't-care mask over disputed regions.
10. Apply augmentation to the training pool.
11. Load best_f1.pth into the model class definitions; fine-tune heads-only at low LR first using the adapted multi-task loss (Stage B); budget for unfreezing deeper layers as the likely next step, not a rare fallback.
12. Evaluate checkpoints against the gold set (extraction-level), then against labtool/benchmark.py (system-level) for TAR/FRR/FAR.
13. Re-calibrate algorithm + threshold; select the deployed checkpoint by real matching accuracy.
14. Run backend/tests/ regression suite.
15. Record the M1 point on the accuracy-vs-data-size curve; decide whether to continue.
16. Continue collection to M2 (~200 people); repeat steps 4–15, warm-starting fine-tuning from the M1 checkpoint.
17. Continue collection to M3 (~300 people) if the curve hasn't plateaued and timeline allows; repeat steps 4–15 again.
18. Run the final testing phase (full cohort accounting, recalibrated algorithm/threshold, explicit gate pass/fail statement).
19. Document results with the same rigor as the existing Internal Testing Annexure before deciding next steps.

Appendix — Ground Truth Labelling: What a Minutia Is

For whoever does the Stage A hand-correction. Set fingerprints aside for a moment — a ridge is just a line, and there are only two things a line does that count as a minutia:

RIDGE ENDING BIFURCATION

Ridge ending — a line runs along with its neighbours, then simply stops. Dead end, like a road with no continuation past that point. **Bifurcation** — the same line, but instead of stopping it forks into two. One line becomes two.

How the labelling actually works

Nobody is asked to find minutiae from a blank image — that's genuinely hard and error-prone even for trained people, especially on noisy contactless photos. The annotator is shown MinutiaeNet's and MINDTCT's candidate points already overlaid on the image, and the job is verification, not invention:

- **Verify, don't invent.** Every point shown was already proposed by one or both extractors. Ask “is this really here,” not “where could one be.”
- **When unsure, skip it.** A missing label costs nothing. A confident wrong one poisons training. If the ridge flow isn't clearly visible — blurred, smudged, shadowed — leave it out rather than force a guess.
- **Direction is data, not decoration.** The direction mark is the ridge's local flow at that exact point. Roughly right is enough — the matcher just needs it close, not exact.

Honest limits, stated plainly

There is no perfect, provable ground truth here. Even professional examiners disagree with each other on ambiguous points in real fingerprint images — this is documented, not a flaw specific to this pipeline. What this process produces is the best available high-confidence consensus, not absolute truth. That's what training data needs to be.

Two practical safeguards worth keeping: a short written reference (a handful of correctly marked example images) before anyone starts, and a second person independently re-checking roughly 10–20% of the gold set to catch systematic errors one annotator alone wouldn't notice — a consistently missed pattern type, or a habitually misread direction.

Prepared as the detailed execution plan for the extraction/fine-tuning lever of the accuracy-improvement effort. Builds on the current cycle's real-data measurements (Internal Testing Annexure) and pipeline research (PIPELINE_ALGORITHMS.md, context.md).